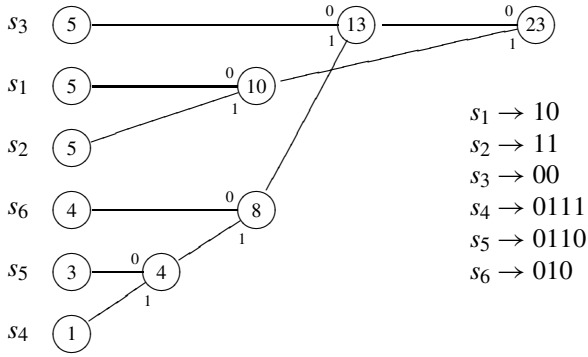
8.1 Adaptive Huffman encoding

In adaptive Huffman encoding, counts of the source letters are kept as scanning of the source text proceeds, and the Huffman tree and corresponding encoding scheme change accordingly. The counts are proportional to the relative frequencies of the source letters in the segment of source text processed so far. (Or, almost proportional—in many cases it is convenient to start with the source letter counts set at some small positive value, like one, which makes the ostensible relative frequencies slightly different from the true relative frequencies. The difference diminishes as you go deeper into the source text. It is possible to start with all counts at zero. The drawback is that the presence of zero node weights complicates the tree management technique due to Knuth, to be described in the next section.) Thus it makes sense to use the counts as weights on the leaf nodes—the nodes associated with the source letters—of the Huffman tree.

Unfortunately, different Huffman trees can be constructed from the same leaf node weights because of choices that sometimes have to be made when two or more nodes carry the same weight. Also, different encoding schemes are associable with the same Huffman tree, because of different choices that can be made in labeling the edges. Therefore, to do adaptive Huffman encoding and decoding, we suffer the annoying necessity of adopting explicit conventions governing how we start and how the choices are to be made in drawing the Huffman trees and labeling their edges. We will also need conventions governing how to go from one Huffman tree to the next after a count has been incremented.

The conventions of this section, to be described below, are not really practical, but will (we think) serve better than "the real thing" as an introduction to adaptive Huffman coding. The real thing, i.e., the actual conventions used in practice are a bit trickier to describe; they are well adapted for computer implementations, but not so easy to walk through in doing pencil and paper exercises. We describe the real conventions, the Knuth-Gallager method of tree management, in Section 8.2.

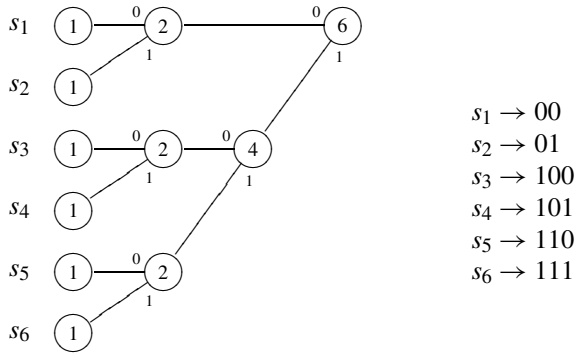
In this section we shall draw our Huffman trees horizontally, as we did in Chapter 5. The weights in the leaf nodes will be non-increasing as you scan from top to bottom. Each leaf node establishes a *level* in the tree; all nodes will be on one of these levels. When two nodes are merged, i.e., when they become siblings, the parent node will be on the higher of the two levels of the sibling nodes. (Thus the root node is guaranteed to be on the level of the highest leaf, not that this is any great advantage.) Here's an example:



This example illustrates two other conventions we will observe: (1) in the labeling of the edges (or "branches") of the tree, the edge going from parent to highest sibling is labeled zero, and the lower edge is labeled 1; (2) we "merge from below in case of ties." For instance, in the construction of the tree above, when it came to pass that the smallest weight of an unmerged node was 5, there were three such nodes to choose among; we chose to merge the two lowest in level.

It remains to specify how to get started, and how the tree will be modified when a count is incremented.

Start: All letters start with a count of 1, and the initial ranking of the source letters, from top to bottom, will be $s_1, \ldots, s_m$. Thus the initial Huffman tree for source text over $S = \{s_1, s_2, s_3, s_4, s_5, s_6\}$ will look like this:



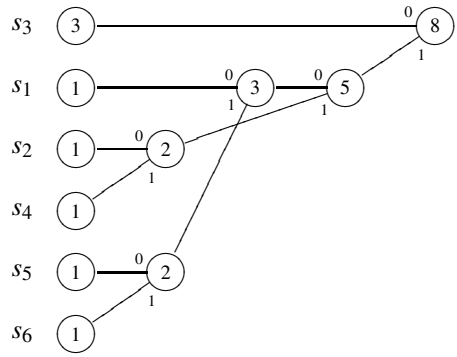
Update after count incrementation: The assignment of leaf nodes to source letters is redone so that the weights are once again in non-increasing order, and the source letter whose count has gone up by one is now attached to the lowest possible node consistent with this monotonicity; the ranking of the other letters is unchanged, but for the possible promotion of that one letter. For instance, in the first example, if any of s_4, s_5, or s_6 is the next letter scanned, the weight in the corresponding leaf node is increased by one and the letter-to-leaf assignments remain the same. In this particular example, incrementing any one of these letter counts does not affect the tree structure, and the encoding scheme remains the

same. If s_2 is scanned, the next tree has its top three leaf nodes assigned to s_2, s_3, and s_1, in that order, with weights 6, 5, and 5, respectively. If s_1 is scanned, the order of the top 3 letters is s_1, s_3, s_2; if s_3 is scanned, the order stays as it is.

Let's try some encoding! We will take $S = \{s_1, s_2, s_3, s_4, s_5, s_6\}$ and source text starting $s_3s_3s_2s_6s_1s_1s_2s_6s_5s_1s_3s_3s_5s_6s_1s_2s_2s_2$. We leave the full encoding of these 18 letters as Exercise 8.1.1, but this is how the code will start:

$$\underset{s_3}{100}\,\underset{s_3}{00}\,\underset{s_2}{110}\,\underset{s_6}{111}\ldots$$

The Huffman tree after the first two letters (s_3's) are scanned looks like this:

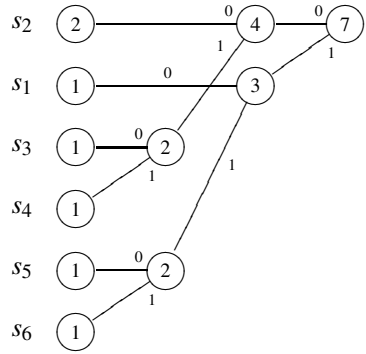


(Are you surprised that the tree comes out like this? Recall the convention that when there is a choice of unmerged nodes with smallest possible weights, merge the two at the lowest possible level.)

Now let's try decoding. With $S = \{s_1, s_2, s_3, s_4, s_5, s_6\}$, and all the conventions mentioned above, suppose the decoder is faced with

$$01100011001011111100\ldots$$

The decoder knows the starting scheme; scanning left to right, the decoder recognizes 01, which is the code word in the starting scheme for s_2. Now the decoder knows what the next Huffman tree will be:



Having recorded s_2, the decoder resumes scanning and soon recognizes 10, the

code word in the current scheme, derived from the tree above, for s_1. The decoder records s_1, makes a new tree, with s_1 now having a count of 2, and forges on.

8.1.1 Compression and readjustment

Keeping letter counts as the source text is scanned amounts to estimating the relative source frequencies by sampling. Therefore, the Huffman tree and associated encoding scheme derived from the letter counts are expected to settle down eventually to the fixed tree and scheme that might have arisen from counting the letters in a large sample of source text before encoding. Because adaptive Huffman encoding is a bit of trouble, and decoding is slow, you might think it is worth the trouble to do a statistical study of the source text first, and proceed with a fixed encoding scheme.

But adaptive Huffman encoding offers an advantage over plain zeroth-order Huffman encoding that can be quite important in situations when the nature of the source text might change. Let us illustrate by taking an extreme case, in which the source text consists of the letter a repeated 10,000 times, then the letter b repeated 10,000 times, then the letter c repeated 10,000 times, and finally the letter d repeated 10,000 times. A thorough statistical study of the source text will reveal that the relative source frequencies are all $1/4$. Plain zeroth-order Huffman encoding will represent a, b, c, and d by the four binary words of length 2. (Of course, first-order Huffman encoding will do very well in this example, but we are trying to compare zeroth-order adaptive and non-adaptive Huffman encoding.)

Now, adaptive Huffman encoding will start well, and, whatever conventions are in force, will encode almost all of those a's by single digits. However, the b's will be encoded with 2 bits each, and then the c's and d's with 3 bits each, squandering the early advantage over static Huffman encoding. Obviously the problem is that when the nature of the source text changes, one or more of the letters may have built up such hefty counts that it takes a long time for the other letters to catch up. To put it another way, the new statistical nature of the source text is obscured by the statistics from the earlier part of the source text.

This defect has a perfectly straightforward remedy, first proposed, we think, by Gallager [23]. The trick is to periodically multiply all the counts by some fraction, like $1/16$, and round down. (If the counts are stored as binary numbers, this operation is particularly easy if the fraction is an integral power of $1/2$.) The beauty of this trick is that if the source text is not in the process of changing its statistical nature, no harm is done, because the new counts after multiplication and rounding are approximately proportional to the old counts; and if the source text is changing, the pile of statistics from earlier text has been reduced from a mountain to a molehill, and it will not take so long for the statistical lineaments of the current text to emerge in the letter counts.

Of course, the decoder must know when and by how much the counts are

to be scaled down. Also, if zero counts are not allowed, then steps might have to be taken to deal with occasional rounding down to zero. But this is a mere annoying detail.

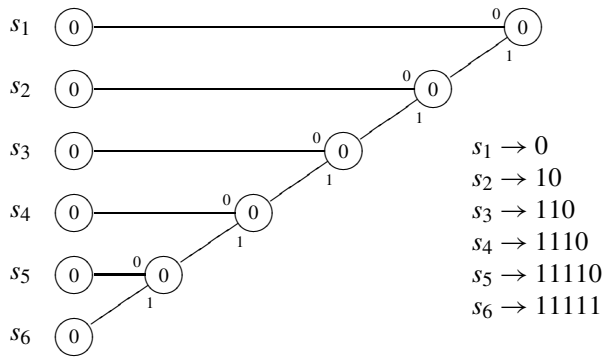
8.1.2 Higher-order adaptive Huffman encoding

For $k \geq 1$, kth-order adaptive Huffman encoding proceeds as you might suppose: for each kth-order context $s_{i_1} \cdots s_{i_k}$ the encoder keeps counts of the source letters occurring in that context (i.e., the scanning of $s_{i_1} \cdots s_{i_k} s_j$ causes the count of s_j in context $s_{i_1} \cdots s_{i_k}$ to increase by 1). Huffman trees are maintained for each context. The encoder and decoder have to agree on conventions for tree formation and maintenance (updating) and also on how to get started; pretty obviously these starting rules will be more extensive than in the zeroth-order case. It seems reasonable to agree to encode the first k letters (before any context is established) according to the zeroth-order adaptive Huffman conventions, whatever they may be.

For example, let us take $S = \{s_1, \ldots, s_6\}$ and the source text we looked at before,

$$s_3 s_3 s_2 s_6 s_1 s_1 s_2 s_6 s_5 s_1 s_3 s_3 s_5 s_6 s_1 s_2 s_2 s_2 \cdots \qquad (*)$$

With tree formation and updating conventions as before, and with all single letter counts starting at 1 (as before) for zeroth-order encoding, let us stipulate that all counts start at 0 in the context schemes; thus, each Huffman tree, in each context, starts off like so:



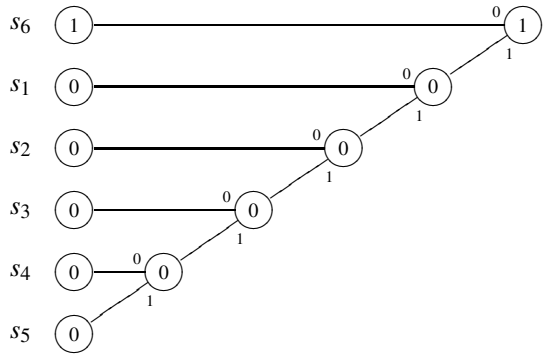
Note that we are allowing zero counts in this instance.

If we attempt second-order adaptive Huffman encoding of $(*)$, with the various conventions and stipulations, the encoding starts

$$\underbrace{100}_{s_3}\,\underbrace{00}_{s_3}\,\underbrace{10}_{s_2}\,\underbrace{11111}_{s_6}\cdots$$

The first two code words are just as before, in the zeroth-order case. After that, the encoding scheme will be stuck on the scheme above, the starting scheme

for all contexts, until a context repeats. In (∗) the first context to repeat is s_2s_6. However, since s_1 followed s_2s_6 the first time s_2s_6 occurred, the encoding scheme for context s_2s_6 will not have changed, and the s_5 that follows its second occurrence will be encoded 11110. (Verify!) By contrast, by the second occurrence of context s_1s_2, the tree for that context is as follows:



Thus the s_2 following the second occurrence of s_1s_2 is encoded 110.

If the source alphabet is quite large, say $m = |S| = 256$, for instance, then maintaining the context trees for $k > 0$ can be expensive. There are m^2 contexts of order 2, and each context tree has m leaf nodes, and that is really a lot of stuff to maintain and to hunt through. Of course, the same volume of stuff has to be kept in *static* (non-adaptive) higher-order Huffman encoding. One way to lessen the burden in static kth-order Huffman encoding is to store the array of $(k+1)$-gram relative frequencies in some sort of alphabetized order so that the line $f(i_1, \ldots, i_k, 1), \ldots, f(i_1, \ldots, i_k, m)$ for context $s_{i_1} \cdots s_{i_k}$ can be quickly found, and then the tree and scheme for that context can be quickly constructed (and stored, if desired) from those relative frequencies. The savings achieved are not great; it does not take much more space to store the static context schemes than it does to store the $(k+1)$-gram relative frequencies.

In adaptive higher-order Huffman encoding we do not have the simplicity of fixed encoding schemes for each context, and it turns out that something similar to the storage procedure suggested above is useful. Letter counts in contexts replace the $(k+1)$-gram relative frequencies. As we shall see in the next section, lists of counts with a few frills and pointers added can function as easily updated Huffman trees/schemes.

There is a further savings possible with adaptive kth-order Huffman (and with adaptive kth-order arithmetic) coding that takes advantage of the fact that some contexts may occur very infrequently. The trick is not to reserve space and maintain counts for a particular context until that context actually occurs. For more detail on the implementation of adaptive higher-order encoding, we refer the interested reader to Bell, Cleary, and Witten [8].

Exercises 8.1

1. Complete the zeroth-order adaptive Huffman encoding of the source text labeled (∗), above, under the conventions of this section.
2. Give the first-order adaptive Huffman encoding of the source text labeled (∗), under the conventions of this section. (In particular: in each context the letter counts start at 0.)
3. Complete the decoding of 01100011001011111100…, with $S = \{s_1, s_2, s_3, s_4, s_5, s_6\}$, under the conventions for zeroth-order adaptive Huffman coding in this section.
4. Decode the fragment of code in problem 3 under the conventions for first-order adaptive Huffman coding in this section (with S as above).

8.2 Maintaining the tree in adaptive Huffman encoding: the method of Knuth and Gallager

The problem with the version of adaptive Huffman encoding presented in Section 8.1 is in the updating of the Huffman tree, after a count is incremented. Sometimes the tree does not change at all, and sometimes it changes drastically. It can change drastically even if the order of assignment of source letters to leaf nodes does not change. There does not seem to be any easy way to see what the next tree will look like; you have to construct the full tree at each stage, after each source letter, and that is a lot of trouble, especially if $m = |S|$ is large.

In practice, the Huffman tree at each stage is a sequence of registers or locations, each representing a node. The contents of each register will be (a) the weight of the node, (b) pointers to the sibling children of that node, or an indication of which source letter the node represents, should it be a leaf, (c) a pointer to the parent of the node, unless the node is the root node, and (d) optionally, an indication, in case the node has sibling children, as to which edge to them is labeled 0, and which is labeled 1. (We will see later why this feature could be optional.)

A tree stored in this way can be used for decoding more easily than the encoding scheme associated with the tree. In order to decode binary code text, start at the root node of the tree; having scanned the first bit, go to the register associated with the sibling child of the root node indicated by that bit, whether 0 or 1. Continue in this way until you arrive at a leaf node; decode the segment of code just scanned as the source letter assigned to that leaf, update the tree, return to the root node, and resume scanning.

Thus, in any Huffman encoding, whether adaptive or not, the Huffman tree can serve efficiently as the Huffman encoding scheme. Now, keeping in mind the sequence-of-registers form of the Huffman tree in storage, we will look at

the efficient tree-updating algorithm due to Knuth [40], based on a mathematical result of Gallager [23]. First, we need to understand Gallager's result.

The tree resulting from an application of Huffman's algorithm to an initial assignment of weights to the leaf nodes belongs to a special class of diagrams called *binary trees*.[1] It is easy to see by induction on m that a binary tree with m leaf nodes has a total of $2m-1$ nodes altogether, and thus $2m-2$ nodes other than the root. (See Exercise 8.2.3.) Gallager's result is the following.

8.2.1 Theorem *Suppose that T is a binary tree with m leaf nodes, $m \geq 2$, with each node u assigned a non-negative weight* $\mathrm{wt}(u)$. *Suppose that each parent is weighted with the sum of the weights of its children. Then T is a Huffman tree (meaning T is obtainable by some instance of Huffman's algorithm applied to the leaf nodes, with their weights), if and only if the $2m-2$ non-root nodes of T can be arranged in a sequence $u_1, u_2, \ldots, u_{2m-2}$ with the properties that*

(a) $\mathrm{wt}(u_1) \leq \mathrm{wt}(u_2) \leq \cdots \leq \mathrm{wt}(u_{2m-2})$ *and*

(b) for $1 \leq k \leq m-1$, u_{2k-1} *and* u_{2k} *are siblings, in T.*

We leave the proof of this theorem as an exercise (see Exercise 8.2.4) for those interested.

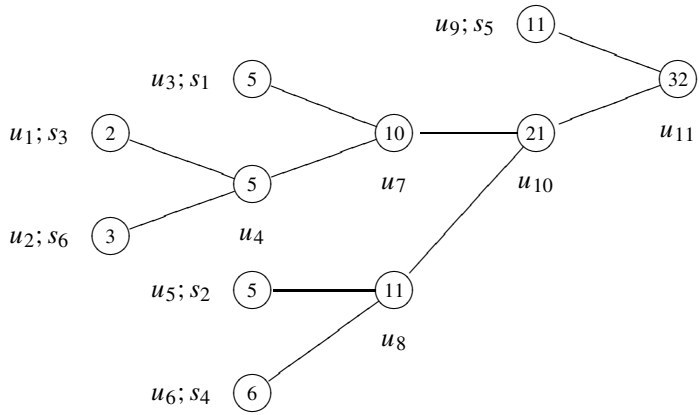
Both Gallager [23] and Knuth [40] propose to manage the Huffman tree at each stage in adaptive Huffman encoding by ordering the nodes $u_1, \ldots, u_{2m-1}$ so that the weight on u_k is non-decreasing with k and so that u_{2k-1} and u_{2k} are siblings in the tree, for $1 \leq k \leq m-1$. (u_{2m-1} will be the root node.) This arrangement allows updating after a count increment in a leaf node in, at worst, a constant times m operations (where m is the number of leaf nodes), while redoing the whole tree from scratch requires a constant times m^2 operations. Plus, you never catch a break redoing the whole tree; it always takes the same number of operations, whereas applying the method of Knuth and Gallager sometimes updates the tree in very few operations.

We refer to *the* method of Knuth *and* Gallager because the two methods are essentially the same, a fact that may not be evident from their descriptions; but the fact is that they always result in the same updated tree (except, possibly, for the leaf labels) by essentially the same steps. Gallager's, the first on the scene, is the slower and more awkward to carry out. Indeed, you can think of Gallager's method as a somewhat inefficient way of carrying out Knuth's algorithm, although the historical truth is that Knuth's algorithm is a clever improvement of Gallager's method. We shall give an account of Gallager's method here as a historical curiosity, and to warm the reader up for Knuth's algorithm, but the reader keen on applications may skip the subsection on Gallager's method.

[1] The technical definition of binary trees is easy but unmemorable: a binary tree is an acyclic connected graph in which exactly one node has degree 2 and all the other nodes have degrees 3 or 1. You can think of a binary tree as the finite diagram obtained by starting at the root node, drawing two edges or branches to two sibling children of the root node, and continuing in this way, deciding for each new node whether it will have two children or remain a childless leaf. The finiteness requirement says that you cannot continue forever allowing some node or other to have children.

In describing the method, we will identify the nodes of the tree with the registers or locations in which information about the nodes are stored. Both versions of the method use *node interchange*, which results in entire subtrees of the current tree being lopped off and regrafted in new positions. It works like this: suppose u_i and u_j are distinct nodes in a node-weighted binary tree. To exchange these nodes, switch the node weights and the names of associated source letters, in case one or both of u_i, u_j are leaf nodes, between the locations. Leave the "forward" pointers to the parents as they are, in locations u_i and u_j. So, effectively, the nodes have exchanged parents; but they keep their children and other descendants, and to effect this you have to go to the registers of the children of u_i (and of u_j), if any, and change their forward pointers so that they point to u_j (resp. u_i). Also, the backward pointers, if any, in u_i and u_j have to switch. (Of course, the names of the nodes are switched as well, so that what was once referred to as u_i is now u_j, but this has nothing to do with what is going on in the registers.)

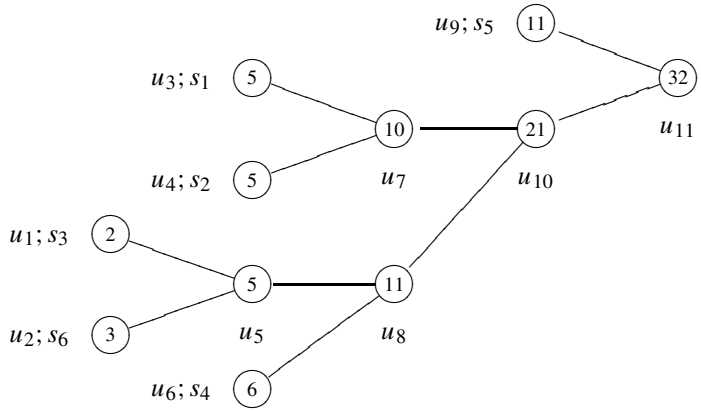
For example, consider the Huffman tree below, lifted from Knuth's paper. Edges will substitute for pointers and the weights are indicated inside the nodes, as usual. The names of the nodes are beside the nodes, as well as the names of the associated source letters, in the case of leaf nodes.



(T_1)

Verify that the weight stored at u_k is a non-decreasing function of k, and that successive nodes u_{2k-1}, u_{2k}, $1 \leq k \leq 5$, in the list $u_1, \ldots, u_{10}$ are siblings.

Now, the interchange of u_4 and u_5 results in the tree (T_2). This also is a Huffman tree, with nodes in the "Gallager order" described in Theorem 8.2.1. It is worth noting that if you start with one Huffman tree with nodes in Gallager order and interchange two nodes, if the two nodes have the same weight then the result will be a Huffman tree with nodes in Gallager order. If the two nodes do not have the same weight then the resulting tree could be a Huffman tree, but, in any case, the nodes will not be in Gallager order; requirement (a) will be no longer satisfied.



(T_2)

Note also the "subtree regrafting" feature of this interchange. The subtree originally rooted at u_4 has been snipped off and regrafted into the tree, now with u_5 as the root. The same is true of the subtree originally rooted at u_5—it is rerooted at u_4—but this rerooting is not very dramatic since that subtree consists of a single node.

8.2.1 Gallager's method

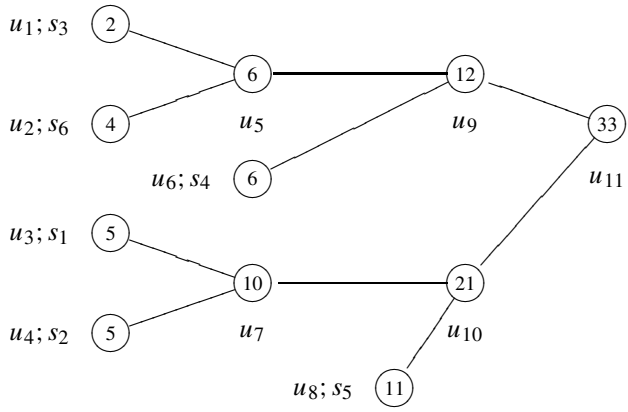
We suppose that the nodes $u_1, \ldots, u_{2m-1}$ of the Huffman tree are stored in Gallager order; for $1 \le k \le m-1$, we refer to u_{2k-1}, u_{2k} as a sibling pair.

Suppose a count $\mathrm{wt}(u)$ in a leaf node u is to be incremented. Change its weight to $\mathrm{wt}(u)+1$ and inspect the next sibling pair up from the one u is in. If the smaller weight in this sibling pair, say on the node v, is smaller than $\mathrm{wt}(u)+1$ (which will be the case if and only if $\mathrm{wt}(u) = \mathrm{wt}(v)$), interchange u and v. In case there is a choice, interchange u with the sibling with larger index. Now again compare $\mathrm{wt}(u)+1$ with the weights in the next sibling pair up (from v). Interchange if $\mathrm{wt}(u)+1$ is larger than one of these. Continue interchanging until the incremented weight, $\mathrm{wt}(u)+1$, is, in fact, smaller than both weights in the sibling pair beyond the node on which that weight now resides, or until there is no sibling pair beyond that node (which will happen if and only if the current residence of $\mathrm{wt}(u)+1$ is a child of the root node).

Suppose the incremented weight is now at node $\widetilde{u}$. Next increment the weight on the parent of $\widetilde{u}$ by one. If the parent of $\widetilde{u}$ is the root node, you are done. Otherwise, proceed with the parent of $\widetilde{u}$ as with u, interchanging until its incremented weight is no greater than that on either node in the next sibling pair up. Then increment its parent—and so on.

For example, suppose the current Huffman tree is T_1, above, and s_6 is scanned. The count in u_2 is increased to 4. The next sibling pair up from u_2 is u_3, u_4, each with weight 5, so we leave location u_2 alone (for now; it will soon have its forward pointers changed) and increment the weight in u_4 by 1; it is now 6. This is greater than the weight in u_5, in the next sibling pair up, so

nodes u_4 and u_5 are interchanged. (So, in particular, nodes u_1 and u_2 are now the children of u_5.) Now the weight 6 in u_5 is less than each weight in u_7 and u_8, corresponding to the next sibling pair up, so increment the weight in u_8 (the parent of u_5) by 1 up to 12. This is greater than the weight in u_9, so exchange u_8 and u_9. Finally, increment the root node. The result of all this:



(T_3)

It is heartily recommended that the reader work through the steps in this example.

8.2.2 Knuth's algorithm

The main differences between the methods of Knuth and Gallager are that Knuth's calls for node interchange *before* any counts are incremented, and there is no emphasis at all on sibling pairs. (Nor did there need to be, really, in Gallager's method.) All node interchanges are of nodes of equal weight, so there need be no interchange of weights between locations. Interchanges involve only changes of pointers and of source letter identities of leaf nodes.

We start with the nodes in Gallager order, $u_1, \ldots, u_{2m-1}$. Suppose that the count on a leaf node u, currently with count $\mathrm{wt}(u)$, is to be incremented. Look up the list from u and find the node $\widetilde{u}$ furthest up the list with the same weight, $\mathrm{wt}(u)$, as u, and interchange u and $\widetilde{u}$. [This $\widetilde{u}$ will be the same node discovered in the first stage of Gallager's method.] Now go to the parent of $\widetilde{u}$. If it is the root node, go to the incrementation phase, described below. Otherwise, find the node the furthest up the list from the parent of $\widetilde{u}$ with the same weight as the parent of $\widetilde{u}$, interchange the parent of $\widetilde{u}$ with that node, proceed to the next parent, and so on.

After all the node interchange is finished, the leaf node now corresponding to the source letter scanned, and each node on the unique path from that node to the root node, have their weights increased by 1, and the update is complete.

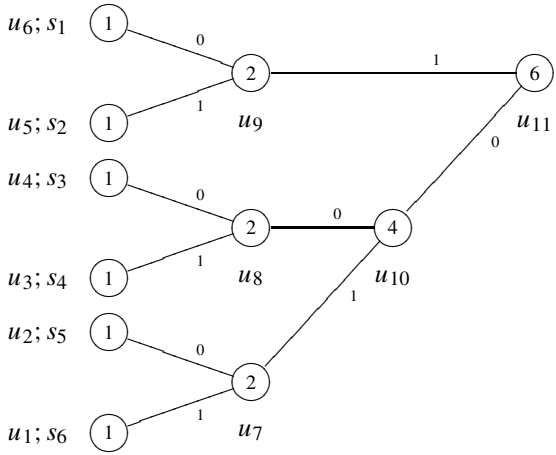
For example, let us again suppose that T_1 is the current state of the Huffman tree, and that s_6 is scanned. The current count 3 in u_2, corresponding to s_6, is

the only 3 appearing, so we move to the next parent node, u_4; u_5 is the location with greatest index (5) bearing the same weight as u_4 (5, by coincidence), so nodes u_4 and u_5 are interchanged. The current picture is then T_2, above.

The next parent to be processed is the node u_8 with weight 11. The highest indexed node with weight 11 is currently u_9, so u_8 and u_9 are interchanged. The next parent is then the root node, so we are done. Now the counts in locations u_2, u_5, u_9, and u_{11} are increased by 1, and we are ready to scan the next letter. Notice that the tree arrived at after incrementation is T_3, the tree arrived at by Gallager's method, and that all the interchanges that took place were between the same pairs of nodes, in the same order, as in Gallager's method. It is always thus, if all weights on the nodes are positive. We leave verification of this statement to the reader.

There is a slight problem with Knuth's algorithm as described when zero counts are allowed on the leaf nodes; it can happen that the u_i with largest index i, with the same weight as the node v you are currently looking at, is u_{2m-1}, the root node. In this eventuality, simply interchange v with u_{2m-2} and proceed to the incrementation stage. By an accident of logic and reference, because of the business about sibling pairs, this provision is built into Gallager's method.

You may be wondering about labeling of the edges of the Huffman tree with 0 or 1, in the Knuth-Gallager procedure. The easiest rule is that if a parent u has children u_{2k-1}, u_{2k}, in the current Gallager ordering of the nodes, then let the edge from u to u_{2k} be labeled 0, and the edge from u to u_{2k-1} be labeled 1. In practice, it might be convenient to have a couple of bits in the register corresponding to u reserved for signaling this labeling, even though it can be figured out from the order of the sibling children in the current Gallager ordering of the nodes.



(T_4)

The Knuth-Gallager procedure requires some sort of initialization convention, because whether the initial counts are set at 1 or 0, there are many choices as to the Gallager order of the nodes at the outset. In the exercise set to follow,

$S = \{s_1, \ldots, s_6\}$, the initial counts are set at 1, and the initial Gallager ordering is indicated by (T_4). Please note that by the edge-labeling convention mentioned above, the edge from u_{11} to u_{10} will be labeled 0.

Exercises 8.2

1. With the initialization and the edge labeling conventions mentioned above, do encoding Exercise 8.1.1 with Knuth's algorithm governing the tree updating.
2. Similarly, do decoding Exercise 8.1.3 with Knuth's algorithm.
3. (a) Show that in a binary tree, there must be at least two leaf nodes that are siblings. [If a binary tree is defined as a tree formed by a certain process, this proposition is evident; the last two children formed will be siblings and leaf nodes.

   Here is another proof. Take a node a greatest distance from the root node. It and its sibling must be leaf nodes. Why?]

   (b) Show that a binary tree with m leaf nodes has $2m - 1$ nodes, total. [Use (a) and go by induction on m.]
4. Prove Theorem 8.2.1. You might follow the following program.

   (a) If T is a binary tree generated by applying Huffman's algorithm to the non-negatively weighted leaf nodes, then the two smallest node weights appear on sibling leaf nodes, by appeal to the procedure of formation. By a similar appeal, the tree obtained by deleting those two leaf nodes, thereby making their parent a leaf, is also a Huffman tree.

   The proof that the nodes of T can be put in Gallager order is now straightforward, by induction on the number of leaf nodes of T.

   (b) Suppose T is a binary tree with non-negatively weighted nodes, with each parent weighted with the sum of the weights of its children. Suppose the nodes of T can be put in Gallager order, $u_1, u_2, \ldots, u_{2m-1}$. If all the weights on nodes are positive, then u_1 and u_2, siblings, must be leaf nodes. (Why?) If zero appears as a weight, then it is possible that one of u_1, u_2 is a parent, say of u_{2k-1}, u_{2k}, $k \geq 2$, but only if the weights on u_{2k-1} and u_{2k} are both zero (verify this assertion, under the assumptions), in which case all the weights on $u_1, \ldots, u_{2k}$ are zero. Switch the sibling pairs u_1, u_2 and u_{2k-1}, u_{2k}, in the ordering; the new ordering is still a Gallager ordering. Switch again, if necessary, and continue switching until the first two nodes in the ordering are sibling leaf nodes. (How can you be sure that all this switching will come to an end with the desired result?) Now consider the tree T' obtained from T by deleting u_1, u_2. Draw the conclusion that T is a Huffman tree, by induction on the number of leaf nodes. (The important formalities are left to you.)